

MANUAL DE GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para desarrolladores en Microsoft Windows con PowerShell 7+, Git Credential Manager y GitHub CLI

[CLI & Web Workflows](#)[SSH & GPG Signing](#)[GitHub Actions CI/CD](#)[Issues & Projects v2](#)[Security & Dependabot](#)[Packages & Releases](#)[API & Scripting](#)

Manual de GitHub: De Novato a Avanzado en PowerShell (Windows)

Plataforma: Microsoft Windows 10 / Windows 11 / Windows Server

Shell: PowerShell 7+ (Core) / Windows PowerShell 5.1 (Windows Terminal)

Herramientas: Git for Windows 2.40+, GitHub CLI (`gh`), Git Credential Manager (GCM), OpenSSH for Windows, Posh-Git

Índice de Contenidos

1. [Parte I: Fundamentos y Configuración del Entorno en Windows](#)
 - 1.1 [Diferencias entre Git y GitHub](#)
 - 1.2 [Instalación con Winget y Optimización de PowerShell](#)
 - 1.3 [Configuración de Identidad y Manejo de CRLF / Rutas Largas](#)
 - 1.4 [Autenticación con Git Credential Manager y GitHub CLI](#)
 - 1.5 [Configuración de OpenSSH en Windows \(Servicio ssh-agent\)](#)
 - 1.6 [Firma Criptográfica de Commits en Windows \(GPG y SSH\)](#)
2. [Parte II: Flujo de Trabajo Esencial \(Nivel Novato\)](#)
 - 2.1 [Creación y Clonación de Repositorios desde PowerShell](#)
 - 2.2 [El Ciclo de Trabajo en Windows: Working Tree, Index y Commit](#)
 - 2.3 [Staging Selectivo y Estándar Conventional Commits](#)
 - 2.4 [Sincronización con GitHub \(`push` , `pull` , `fetch` \)](#)
 - 2.5 [Control de Archivos Ignorados en Windows \(`.gitignore` \)](#)
3. [Parte III: Ramas, Fusiones y Estrategias Colaborativas \(Nivel Intermedio\)](#)
 - 3.1 [Gestión de Ramas con PowerShell y Posh-Git](#)
 - 3.2 [Los Tres Métodos de Merge en GitHub](#)
 - 3.3 [Modelos de Flujo: GitHub Flow, Git Flow y Forking](#)
 - 3.4 [Operaciones Avanzadas: Stash, Cherry-Pick y Rebase Interactivo](#)
 - 3.5 [El Salvavidas: Recuperación con `git reflog`](#)
4. [Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo](#)
 - 4.1 [Tema 1: Prevención y Buenas Prácticas de Equipo](#)
 - 4.2 [Tema 2: Fusión Automática \(Cambios en Distintas Líneas\)](#)
 - 4.3 [Tema 3: Conflicto Directo de Fusión \(Mismas Líneas\) con VS Code](#)
 - 4.4 [Tema 4: Elección Total de Versión \(`--ours` vs `--theirs` \)](#)

- 4.5 [Tema 5: Cambios Locales sin Confirmar al hacer Pull \(`git stash` \)](#)
 - 4.6 [Tema 6: Push Rechazado por Desfase \(`non-fast-forward` \) y Rebase Seguro](#)
 - 4.7 [Tema 7: Resolución de Conflictos en Pull Requests \(Web y CLI\)](#)
 - 4.8 [Tema 8: Conflicto de Modificación vs Eliminación](#)
 - 4.9 [Tema 9: Archivos Binarios y Bloqueo con Git LFS en Windows](#)
5. [Parte V: Gestión de Proyectos y Ecosistema GitHub](#)
- 5.1 [GitHub Issues, Milestones y Labels desde PowerShell](#)
 - 5.2 [Pull Requests y Revisiones desde GitHub CLI](#)
 - 5.3 [GitHub Projects \(v2\): Automatizaciones y Tableros](#)
 - 5.4 [GitHub Discussions y Wikis en Windows](#)
6. [Parte VI: Automatización y CI/CD con GitHub Actions \(Nivel Avanzado\)](#)
- 6.1 [Estructura de Pipelines y Sintaxis YAML](#)
 - 6.2 [Workflows con Runners \(`windows-latest` \) y Shell \(`pwsh` \)](#)
 - 6.3 [Secretos, Matrices Multiplataforma y Caching](#)
 - 6.4 [Configuración de un Self-Hosted Runner como Servicio de Windows](#)
7. [Parte VII: Distribución, Paquetes y Publicación](#)
- 7.1 [GitHub Releases: Binarios \(`.exe` , `.msi` \) y Archivos \(`.zip` \)](#)
 - 7.2 [GitHub Packages: Registro de Paquetes NuGet y npm](#)
 - 7.3 [GitHub Pages: Publicación Automatizada](#)
8. [Parte VIII: Seguridad, Gobernanza y Administración](#)
- 8.1 [Reglas de Protección de Ramas y Rulesets](#)
 - 8.2 [Dependabot, Secret Scanning y Push Protection](#)
 - 8.3 [Análisis de Seguridad SAST con CodeQL](#)
 - 8.4 [Gobernanza con \(`CODEOWNERS` \) y Permisos](#)
9. [Parte IX: Scripting PowerShell con la API y Diagnóstico](#)
- 9.1 [Consultas a la API REST con \(`gh api` \) y \(`ConvertFrom-Json` \)](#)
 - 9.2 [Automatización con Scripts de PowerShell \(`.ps1` \)](#)
 - 9.3 [Diagnóstico y Resolución de Problemas Típicos de Windows](#)
-

Parte I: Fundamentos y Configuración del Entorno en Windows

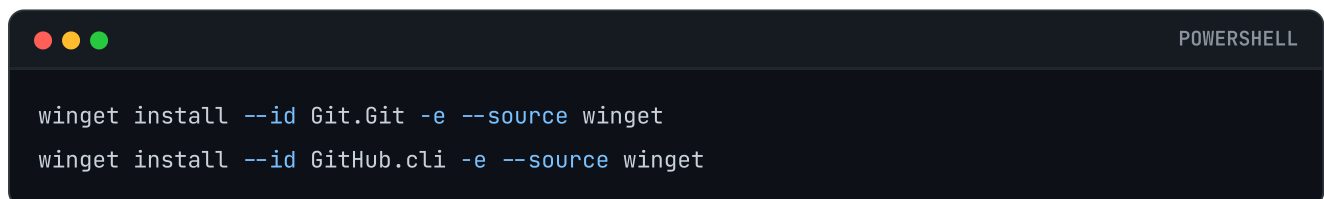
1.1 Diferencias entre Git y GitHub

- **Git:** Motor de control de versiones distribuido que opera localmente en Windows. Gestiona el historial de archivos mediante instantáneas criptográficas dentro de la carpeta oculta `.git` sin requerir conexión a la red.
- **GitHub:** Plataforma en la nube de desarrollo colaborativo que centraliza repositorios remotos, facilita revisiones de código mediante Pull Requests, integra automatización con GitHub Actions y provee herramientas de seguridad avanzada.

1.2 Instalación con Winget y Optimización de PowerShell

Abre una ventana de **PowerShell como Administrador**:

Paso 1: Instalar Git y GitHub CLI mediante Winget



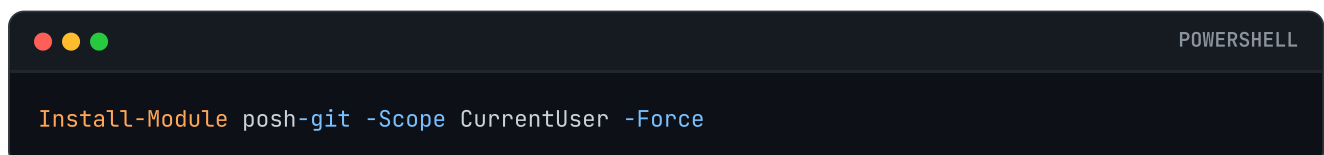
```
POWERSHELL

winget install --id Git.Git -e --source winget
winget install --id GitHub.cli -e --source winget
```

¿Qué hace este comando?

Descarga e instala los paquetes oficiales de 64 bits para Windows de Git y GitHub CLI sin instaladores manuales.

Paso 2: Instalar Posh-Git (autocompletado y estado de Git en el prompt)



```
POWERSHELL

Install-Module posh-git -Scope CurrentUser -Force
```

¿Qué hace este comando?

Instala el módulo de PowerShell `posh-git`, que añade autocompletado avanzado y muestra información de ramas y cambios en tu línea de comandos.

Paso 3: Configurar carga automática en el perfil de PowerShell

```
POWERSHELL

if (!(Test-Path -Path $PROFILE)) { New-Item -ItemType File -Path $PROFILE -Force }
Add-Content -Path $PROFILE -Value "Import-Module posh-git"
& $PROFILE
```

¿Qué hace este comando?

Comprueba si existe tu script de perfil personal (`$PROFILE`), lo crea si es necesario, agrega la importación de `posh-git` y lo recarga inmediatamente en la sesión actual.

1.3 Configuración de Identidad y Manejo de CRLF / Rutas Largas

Configurar nombre de autor y correo

```
POWERSHELL

git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu-correo@ejemplo.com"
```

¿Qué hace este comando?

Establece tu identidad global para todas las confirmaciones (commits) que realices en tu equipo.

Configurar saltos de línea (CRLF) y soporte de rutas largas

```
POWERSHELL

git config --global core.autocrlf true
git config --global core.longpaths true
git config --global init.defaultBranch main
```

¿Qué hace este comando?

- `core.autocrlf true` : Convierte los saltos de línea CRLF de Windows a LF al enviar a GitHub, y de vuelta a CRLF al descargar a tu disco.
- `core.longpaths true` : Elimina el límite histórico de 260 caracteres de Windows en las rutas de archivo de repositorios profundos.
- `init.defaultBranch main` : Establece `main` como la rama por defecto al inicializar repositorios.

1.4 Autenticación con Git Credential Manager y GitHub CLI

```
git config --global credential.helper manager
gh auth login --hostname github.com --git-protocol https --web
```

¿Qué hace este comando?

Activa el ayudante oficial **Git Credential Manager (GCM)**, que guarda las credenciales en el Administrador de Credenciales de Windows, e inicia sesión con autenticación web segura (compatible con 2FA).

```
gh auth status
```

¿Qué hace este comando?

Muestra el estado actual de la sesión iniciada en GitHub CLI, indicando el usuario activo y los permisos concedidos.

1.5 Configuración de OpenSSH en Windows (Servicio ssh-agent)

Paso 1: Habilitar el servicio ssh-agent de Windows

```
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent
```

¿Qué hace este comando?

Configura el servicio nativo OpenSSH de Windows para que arranque automáticamente con el sistema y lo inicia en este momento.

Paso 2: Generar y cargar clave Ed25519

```
ssh-keygen -t ed25519 -C "tu-correo@ejemplo.com" -f "$HOME\.ssh\id_ed25519"
ssh-add "$HOME\.ssh\id_ed25519"
```

¿Qué hace este comando?

Genera un par de claves seguras en tu directorio de usuario y carga la clave privada en el agente de Windows.

Paso 3: Configurar host y asociar clave a GitHub

```
POWERSHELL

$sshConfig = @"
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
"@
Set-Content -Path "$HOME\.ssh\config" -Value $sshConfig
gh ssh-key add "$HOME\.ssh\id_ed25519.pub" --title "Windows-PowerShell-PC"
ssh -T git@github.com
```

¿Qué hace este comando?

Crea el archivo de configuración SSH para conectar a GitHub, sube automáticamente tu clave pública a tu perfil mediante `gh` y comprueba la conexión.

1.6 Firma Criptográfica de Commits en Windows (GPG y SSH)

```
POWERSHELL

git config --global gpg.format ssh
git config --global user.signingkey "$HOME/.ssh/id_ed25519.pub"
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

¿Qué hace este comando?

Activa la firma digital automática en todos tus commits y etiquetas utilizando tu clave pública SSH de Windows para obtener la insignia **Verified** en GitHub.

Parte II: Flujo de Trabajo Esencial (Nivel Novato)

2.1 Creación y Clonación de Repositorios desde PowerShell

Caso A: Crear un repositorio local y subirlo a GitHub

```
New-Item -ItemType Directory -Path ".\mi-proyecto-windows" -Force
Set-Location ".\mi-proyecto-windows"
git init
Set-Content -Path "README.md" -Value "# Proyecto Windows en GitHub"
git add README.md
git commit -m "docs: inicializar repositorio"
gh repo create mi-proyecto-windows --public --source=. --remote=origin --push
```

¿Qué hace este comando?

Crea la carpeta, inicia el repositorio local, genera el `README.md`, realiza el commit inicial y crea el repositorio remoto público en GitHub vinculándolo y enviando los archivos en una sola instrucción.

Caso B: Clonar un repositorio existente

```
git clone git@github.com:usuario/mi-repositorio.git
```

¿Qué hace este comando?

Descarga el repositorio remoto y todo su árbol de commits a tu equipo mediante SSH.

2.2 El Ciclo de Trabajo en Windows: Working Tree, Index y Commit

```
git status
```

¿Qué hace este comando?

Muestra qué archivos han sido modificados, cuáles están listos para commit en el área de preparación (staging) y cuáles no tienen seguimiento.

```
git diff
```

¿Qué hace este comando?

Compara los cambios que tienes en tus archivos en disco contra el área de preparación.

```
git diff --staged
```

¿Qué hace este comando?

Inspecciona las diferencias de los cambios que ya están preparados en el staging.

2.3 Staging Selectivo y Estándar Conventional Commits

```
git add -p .\Controlador.cs
```

¿Qué hace este comando?

Permite preparar de forma interactiva fragmentos específicos de un archivo, seleccionando qué bloques enviar al commit y cuáles reservar.

Formato Conventional Commits

Tipo	Finalidad	Ejemplo en PowerShell
feat:	Nueva funcionalidad	<code>git commit -m "feat(ui): agregar soporte para modo oscuro"</code>
fix:	Corrección de fallo	<code>git commit -m "fix(io): resolver bloqueo de archivos en Windows"</code>
docs:	Modificación de documentación	<code>git commit -m "docs: actualizar requisitos de PowerShell"</code>
refactor:	Refactorización interna	<code>git commit -m "refactor: optimizar consulta LINQ"</code>
test:	Cobertura de pruebas	<code>git commit -m "test: añadir pruebas unitarias en Windows"</code>
chore:	Tareas de mantenimiento	<code>git commit -m "chore: actualizar paquetes NuGet"</code>

2.4 Sincronización con GitHub (push, pull, fetch)

POWERSHELL

```
git fetch origin
```

¿Qué hace este comando?

Descarga la información y ramas más recientes de GitHub sin tocar tus archivos de trabajo.

POWERSHELL

```
git pull --rebase origin main
```

¿Qué hace este comando?

Descarga los commits nuevos de GitHub y coloca tus commits locales por encima de ellos, evitando ramas divididas y commits de merge innecesarios.

POWERSHELL

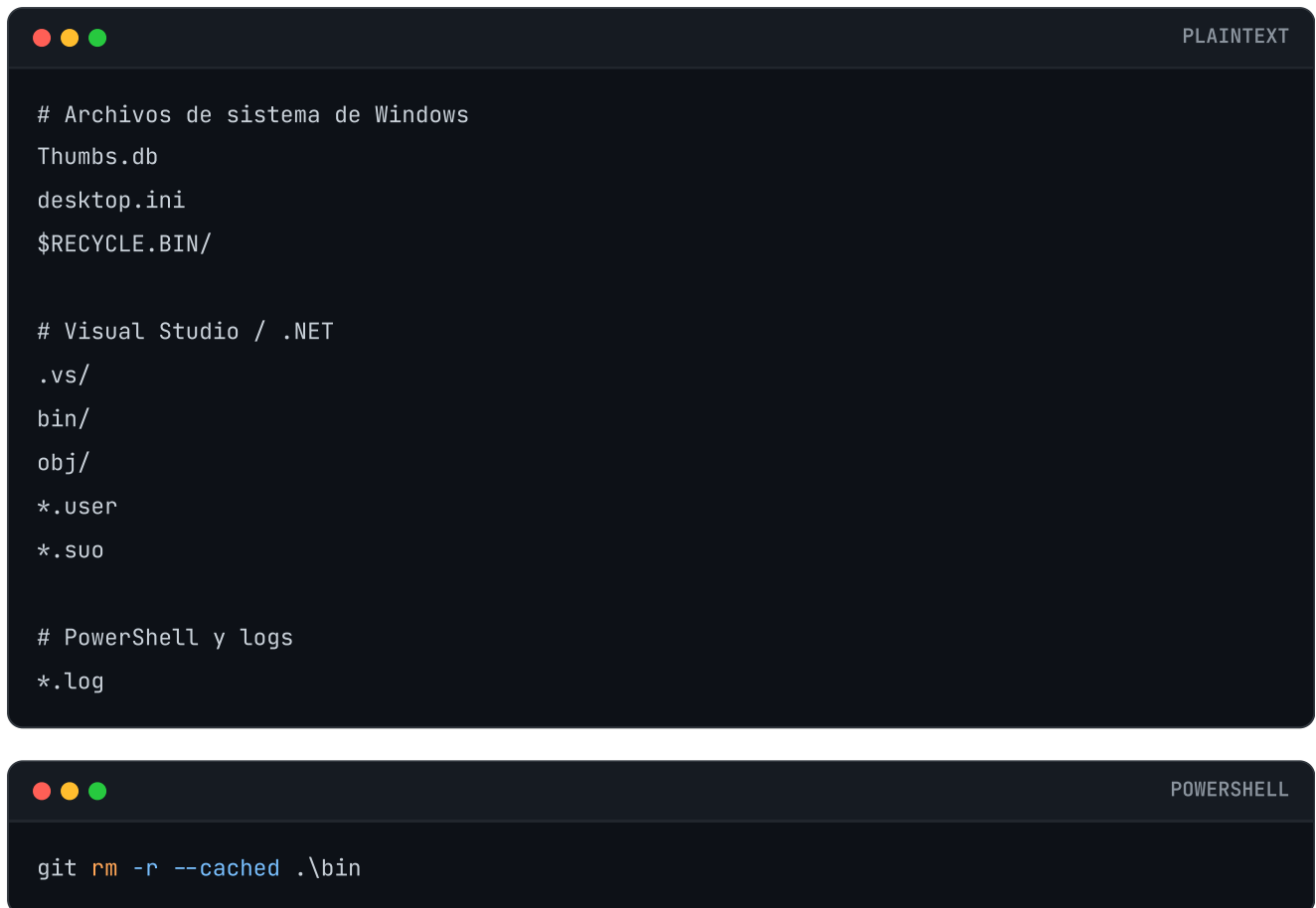
```
git push -u origin main
```

¿Qué hace este comando?

Envía tus confirmaciones locales a GitHub y asocia la rama para futuros comandos `git push`.

2.5 Control de Archivos Ignorados en Windows (.gitignore)

Crea un archivo `.gitignore` con archivos comunes de Windows y Visual Studio:



¿Qué hace este comando?

Elimina del control de versiones de Git una carpeta que fue commiteada por error sin borrar los archivos físicos de tu disco.

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Gestión de Ramas con PowerShell y Posh-Git



```
git switch -c feature/conexion-sql
```

POWERSHELL

¿Qué hace este comando?

Crea una nueva rama de trabajo llamada `feature/conexion-sql` y cambia inmediatamente a ella.



```
git push -u origin feature/conexion-sql
```

POWERSHELL

¿Qué hace este comando?

Publica la rama local en GitHub y configura el rastreo remoto.



```
git switch main  
git branch -d feature/conexion-sql  
git push origin --delete feature/conexion-sql
```

POWERSHELL

¿Qué hace este comando?

Regresa a la rama principal, borra la rama local ya integrada (`-d`) y elimina la rama en GitHub.

3.2 Los Tres Métodos de Merge en GitHub

- **Merge Commit:** Fusión clásica que preserva el historial exacto de cada commit y crea un commit conmemorativo de unión.
- **Squash and Merge:** Aplasta todos los commits de la rama en un único commit conciso sobre la rama principal.
- **Rebase and Merge:** Coloca los commits individualmente al final de la rama destino, logrando una historia completamente lineal.

3.3 Modelos de Flujo: GitHub Flow, Git Flow y Forking

Forking Workflow en Windows

```
POWERSHELL

# 1. Crear fork y clonarlo
gh repo fork microsoft/PowerShell --clone
Set-Location .\PowerShell

# 2. Sincronizar el fork con el repositorio upstream original
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

¿Qué hace este comando?

Crea una bifurcación (fork) en tu cuenta de GitHub, la descarga a tu equipo y mantiene sincronizada tu rama local con los cambios originales.

3.4 Operaciones Avanzadas: Stash, Cherry-Pick y Rebase Interactivo

```
POWERSHELL

git stash save "Trabajo preliminar sin terminar"
```

¿Qué hace este comando?

Guarda las modificaciones no confirmadas en la pila de stash y limpia el directorio de trabajo.

```
POWERSHELL

git stash pop
```

¿Qué hace este comando?

Reubica los cambios guardados del último stash en tus archivos de trabajo.

```
POWERSHELL

git cherry-pick 4f2a1b9
```

¿Qué hace este comando?

Copia y aplica un commit específico de otra rama sobre la rama en la que estás ubicado.

POWERSHELL

```
git rebase -i HEAD~4
```

¿Qué hace este comando?

Inicia un rebase interactivo sobre los últimos 4 commits locales para reordenar, fusionar o modificar mensajes antes de subir a GitHub.

3.5 El Salvavidas: Recuperación con `git reflog`

POWERSHELL

```
git reflog
```

¿Qué hace este comando?

Consulta el historial de todos los movimientos de `HEAD` en tu máquina.

POWERSHELL

```
git reset --hard "HEAD@{2}"
```

¿Qué hace este comando?

Revierte de forma segura y exacta el repositorio al estado en que se encontraba hace dos operaciones.

Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo

Cuando dos personas modifican el mismo archivo dentro de una misma carpeta en un proyecto colaborativo, pueden darse diferentes situaciones. A continuación se presentan las soluciones según cada tema:

4.1 Tema 1: Prevención y Buenas Prácticas de Equipo

1. **Trabajar en ramas independientes:** Ningún desarrollador debe realizar commits directos a `main`. Cada funcionalidad debe vivir en `feature/nombre-tarea`.
2. **Modularización:** Separar archivos extensos en componentes independientes para reducir la probabilidad de colisiones.
3. **Draft Pull Requests:** Crear PRs tempranos en modo borrador para que el equipo visualice qué archivos están bajo edición activa.
4. **Descargas frecuentes:** Ejecutar `git pull --rebase origin main` periódicamente para no desfasarse de los cambios de los demás.

4.2 Tema 2: Fusión Automática (Cambios en Distintas Líneas)

Si tú y tu compañero tocaron secciones diferentes del mismo archivo, Git es capaz de combinar los cambios de forma automática sin generar conflictos:

```
POWERSHELL

# 1. Descargar e incorporar los cambios del compañero aplicando rebase
git pull --rebase origin main
```

¿Qué hace este comando?

Trae el commit de tu compañero y reaplica tus cambios encima. Git mostrará:

```
Auto-merging src/Controlador.cs
```

```
Applying: feat: mi cambio local
```

```
# 2. Subir tus cambios integrados a GitHub  
git push origin main
```

4.3 Tema 3: Conflicto Directo de Fusión (Mismas Líneas) con VS Code

Si ambos editaron exactamente las mismas líneas, Git marcará un conflicto. Configuraremos Visual Studio Code como herramienta visual oficial de merge:

Paso 1: Configurar VS Code como visor de diferencias y fusión

```
git config --global diff.tool vscode  
git config --global difftool.vscode.cmd 'code --wait --diff $LOCAL $REMOTE'  
git config --global merge.tool vscode  
git config --global mergetool.vscode.cmd 'code --wait $MERGED'
```

Paso 2: Ante un conflicto, abrir la herramienta visual de merge

```
git mergetool
```

¿Qué hace este comando?

Abre automáticamente Visual Studio Code con el editor de fusión de 3 vías (3-way merge editor), permitiéndote hacer clic en Accept Current, Accept Incoming o Accept Both Changes con un clic.

Paso 3: Guardar y finalizar la operación

- Si estabas en un merge:

```
git add .\Controlador.cs  
git commit -m "merge: resolver conflicto en Controlador.cs"  
git push origin main
```

- Si estabas en un rebase:

```
git add .\Controlador.cs
git rebase --continue
git push origin main
```

TIP

Para abortar y regresar al estado limpio anterior al conflicto:

```
git merge --abort
# 0 bien:
git rebase --abort
```

4.4 Tema 4: Elección Total de Versión (--ours vs --theirs)

Si no deseas mezclar líneas y una de las dos versiones es la definitiva:

Conservar tu versión completa:

```
git checkout --ours .\Controlador.cs
git add .\Controlador.cs
git commit -m "resolve: conservar versión local de Controlador.cs"
git push origin main
```

Aceptar la versión del compañero en su totalidad:

```
git checkout --theirs .\Controlador.cs
git add .\Controlador.cs
git commit -m "resolve: aceptar versión remota de Controlador.cs"
git push origin main
```

4.5 Tema 5: Cambios Locales sin Confirmar al hacer Pull (`git stash`)

Si intentas hacer `git pull` mientras tienes cambios sin commitear en el archivo que tu compañero subió:

```
POWERSHELL

# 1. Apartar tus cambios locales en el stash
git stash save "Cambios en progreso antes de pull"

# 2. Descargar los cambios de GitHub
git pull --rebase origin main

# 3. Volcar nuevamente tus cambios
git stash pop
```

¿Qué hace este comando?

Guarda tus modificaciones locales en un búfer temporal, actualiza el repositorio con la versión de GitHub y reaplica tus cambios.

4.6 Tema 6: Push Rechazado por Desfase (`non-fast-forward`) y Rebase Seguro

Si tu intento de `git push` es rechazado con el error `failed to push some refs`:

```
POWERSHELL

# 1. NUNCA fuerces con git push --force (borrarías los cambios de tu compañero)

# 2. Sincroniza y sitúa tus commits sobre los suyos:
git pull --rebase origin main

# 3. Tras reordenar commits (o resolver conflictos), sube limpiamente:
git push origin main
```

4.7 Tema 7: Resolución de Conflictos en Pull Requests (Web y CLI)

Desde la Web de GitHub:

1. Abre el Pull Request en tu navegador.

2. Si hay colisión, haz clic en **Resolve conflicts**.
3. Elimina las marcas `<<<<<<` y `>>>>>>`, selecciona el código final y haz clic en **Commit merge**.

Desde PowerShell con GitHub CLI (gh):

```
POWERSHELL

# Descargar la rama del PR
gh pr checkout 18

# Fusionar la rama principal actualizada para reproducir el conflicto
git merge origin/main

# Resolver en VS Code y confirmar
code .\Controlador.cs
git add .\Controlador.cs
git commit -m "merge: sincronizar con main y resolver colisiones"
git push origin HEAD
```

4.8 Tema 8: Conflicto de Modificación vs Eliminación

Ocurre cuando una persona editó el archivo y la otra lo borró.

- Si deseas conservar el archivo:

```
POWERSHELL

git add .\Controlador.cs
git commit -m "resolve: mantener archivo modificado"
```

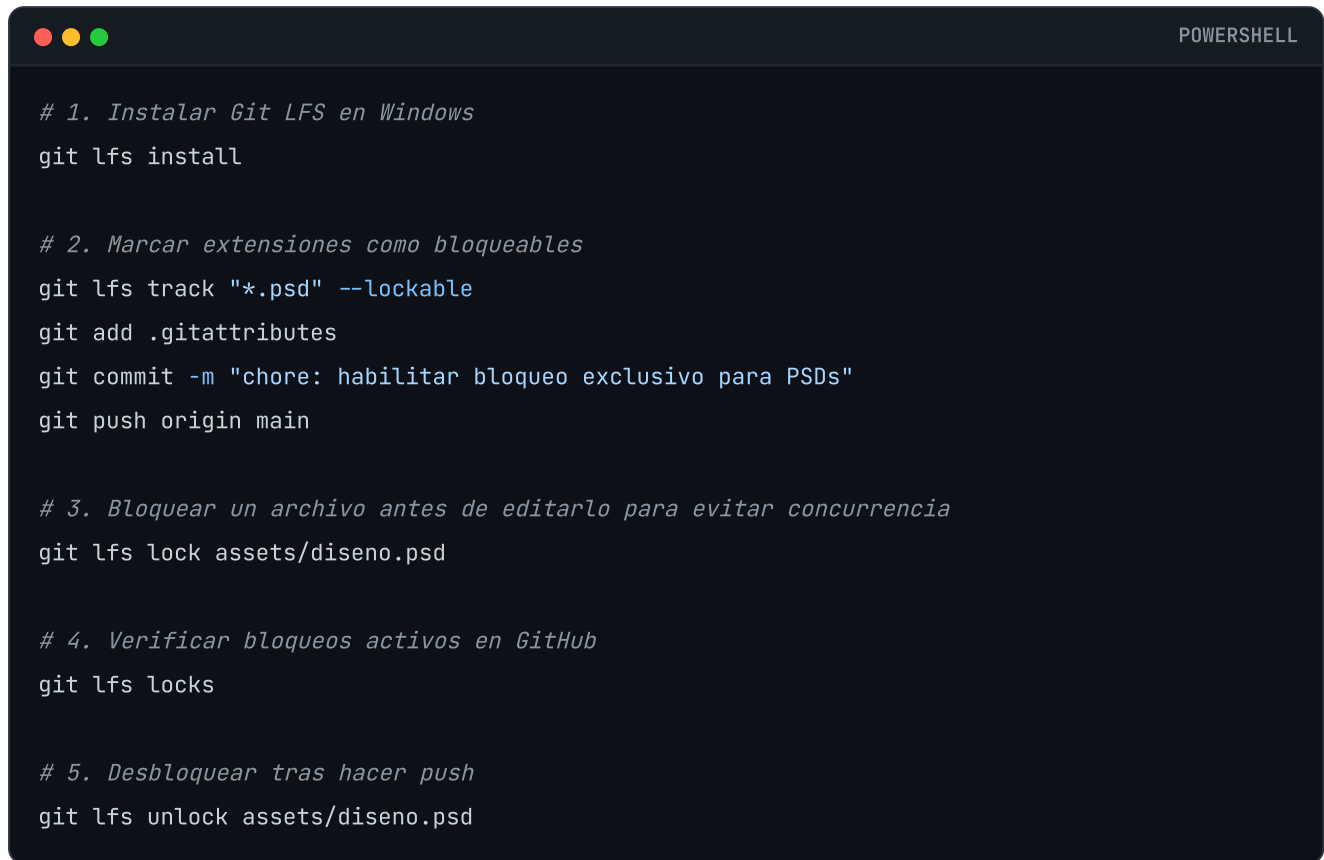
- Si aceptas eliminarlo:

```
POWERSHELL

git rm .\Controlador.cs
git commit -m "resolve: confirmar eliminación del archivo"
```

4.9 Tema 9: Archivos Binarios y Bloqueo con Git LFS en Windows

Los archivos como instaladores `.exe`, bibliotecas `.dll`, imágenes o bases de datos no se pueden mezclar con diferencias de texto.



```
POWERSHELL

# 1. Instalar Git LFS en Windows
git lfs install

# 2. Marcar extensiones como bloqueables
git lfs track "*.psd" --lockable
git add .gitattributes
git commit -m "chore: habilitar bloqueo exclusivo para PSDs"
git push origin main

# 3. Bloquear un archivo antes de editarlo para evitar concurrencia
git lfs lock assets/disenio.psd

# 4. Verificar bloqueos activos en GitHub
git lfs locks

# 5. Desbloquear tras hacer push
git lfs unlock assets/disenio.psd
```

Parte V: Gestión de Proyectos y Ecosistema GitHub

5.1 GitHub Issues, Milestones y Labels desde PowerShell

POWERSHELL

```
gh issue create --title "Excepción en conexión SQL" --body "Error de timeout bajo Windows Server 2022." --label "bug,windows"
```

¿Qué hace este comando?

Crea una incidencia en GitHub con título, descripción y etiquetas desde PowerShell.

POWERSHELL

```
gh issue list --assignee "@me"
```

¿Qué hace este comando?

Muestra en una tabla los issues asignados a ti.

POWERSHELL

```
gh issue close 14 --comment "Solucionado con el nuevo pool de conexiones."
```

¿Qué hace este comando?

Cierra formalmente el issue #14 con un comentario aclaratorio.

5.2 Pull Requests y Revisiones desde GitHub CLI

POWERSHELL

```
gh pr create --title "feat: módulo de compresión ZIP" --body "Implementación nativa con System.IO.Compression" --assignee "@me"
```

¿Qué hace este comando?

Crea un Pull Request formal en GitHub listo para ser revisado por tus compañeros.

```
gh pr checkout 8
```

¿Qué hace este comando?

Conmuta tu entorno local de Windows a la rama del Pull Request #8 para depurarlo.

```
gh pr review 8 --approve -b "Validado en Windows 11."
```

¿Qué hace este comando?

Emite un dictamen de aprobación formal sobre el Pull Request.

5.3 GitHub Projects (v2): Automatizaciones y Tableros

```
gh project list  
gh project item-add 5 --owner "mi-empresa" --url "https://github.com/usuario/repo/issues/14"
```

¿Qué hace este comando?

Lista tus tableros y asocia un issue directamente a la columna correspondiente de tu proyecto ágil.

5.4 GitHub Discussions y Wikis en Windows

```
git clone git@github.com:usuario/mi-proyecto-windows.wiki.git  
Set-Location .\mi-proyecto-windows.wiki  
Add-Content -Path "Guia-PowerShell.md" -Value "## Configuración recomendada"  
git add Guia-PowerShell.md  
git commit -m "docs: añadir guía de PowerShell a la wiki"  
git push origin master
```

¿Qué hace este comando?

Clona la wiki del repositorio, añade una página en Markdown y la publica mediante Git.

Parte VI: Automatización y CI/CD con GitHub Actions (Nivel Avanzado)

6.1 Estructura de Pipelines y Sintaxis YAML

Los workflows se definen en formato YAML bajo `.github/workflows/*.yaml` y se ejecutan ante eventos configurables.

6.2 Workflows con Runners `windows-latest` y Shell `powershell`

Crea `.github/workflows/windows-ci.yaml` :

```
name: Windows .NET & PowerShell CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: windows-latest
    defaults:
      run:
        shell: pwsh

    steps:
      - name: Descargar código
        uses: actions/checkout@v4

      - name: Configurar SDK de .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'

      - name: Restaurar dependencias
        run: dotnet restore

      - name: Compilar solución en modo Release
        run: dotnet build --configuration Release --no-restore

      - name: Ejecutar pruebas unitarias
        run: dotnet test --no-build --verbosity normal --logger
"trx;LogFileName=test_results.trx"

      - name: Empaquetar artefactos de compilación
        run: |
          Compress-Archive -Path .\bin\Release\net8.0\* -DestinationPath .\App-Windows-
x64.zip

      - name: Subir artefacto compilado
        uses: actions/upload-artifact@v4
```

```
with:
  name: windows-application-binary
  path: .\App-Windows-x64.zip
```

6.3 Secretos, Matrices Multiplataforma y Caching

```
strategy:
  matrix:
    os: [windows-latest, ubuntu-latest]
    dotnet: ['7.0.x', '8.0.x']
```

6.4 Configuración de un Self-Hosted Runner como Servicio de Windows

Abre PowerShell como Administrador:

```
# 1. Crear carpeta del runner
New-Item -ItemType Directory -Path "C:\actions-runner" -Force
Set-Location "C:\actions-runner"

# 2. Descargar el software oficial para Windows x64
Invoke-WebRequest -Uri
"https://github.com/actions/runner/releases/download/v2.316.0/actions-runner-win-x64-2.316.0.zip" -OutFile "runner.zip"
Expand-Archive -Path "runner.zip" -DestinationPath "."

# 3. Registrar con el token de GitHub
.\config.cmd --url https://github.com/usuario/repo --token TU_TOKEN_AQUI

# 4. Instalar y arrancar como Servicio de Windows
.\svc.cmd install
.\svc.cmd start
Get-Service "actions.runner.*"
```

Parte VII: Distribución, Paquetes y Publicación

7.1 GitHub Releases: Binarios .exe, .msi y Archivos .zip

```
# 1. Crear y subir un tag firmado
git tag -a v2.0.0 -m "release: versión 2.0.0 estable"
git push origin v2.0.0

# 2. Publicar Release y adjuntar los instaladores
gh release create v2.0.0 .\Instalador.msi .\App.zip `
  --title "Versión 2.0.0 para Windows" `
  --generate-notes
```

¿Qué hace este comando?

Publica formalmente la versión v2.0.0 en GitHub, genera las notas automáticas de cambios y adjunta los instaladores para que los usuarios puedan descargarlos.

7.2 GitHub Packages: Registro de Paquetes NuGet y npm

```
# Registrar la fuente de GitHub en NuGet
dotnet nuget add source "https://nuget.pkg.github.com/usuario/index.json" `
  -n "GitHubPackages" `
  -u "TU_USUARIO" `
  -p "TU_TOKEN_PAT" `
  --store-password-in-clear-text

# Publicar el paquete NuGet
dotnet nuget push .\bin\Release\MiLibreria.1.0.0.nupkg --source "GitHubPackages"
```

7.3 GitHub Pages: Publicación Automatizada

Alojamiento estático para sitios de documentación desplegados automáticamente con GitHub Actions.

Parte VIII: Seguridad, Gobernanza y Administración

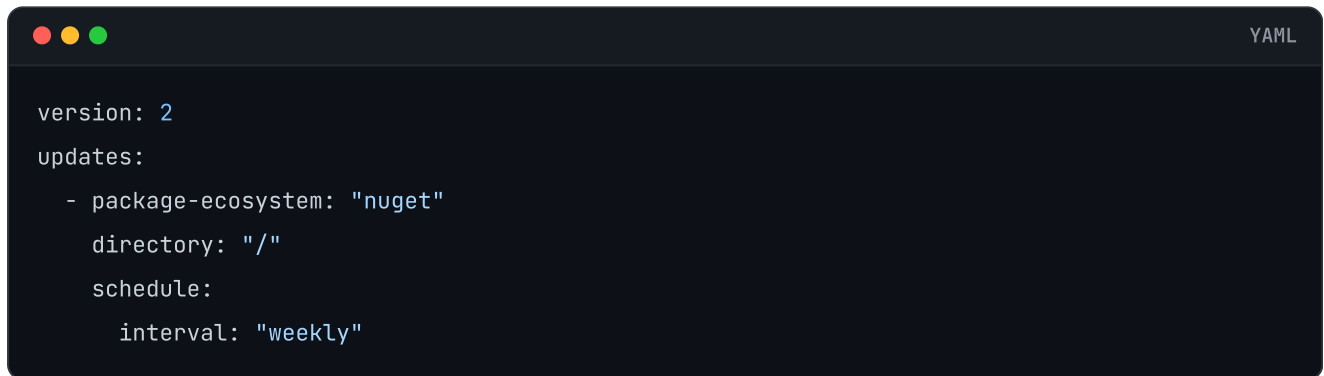
8.1 Reglas de Protección de Ramas y Rulesets

Desde **Settings** → **Rules** → **Rulesets**:

- Bloquear commits directos a `main`.
- Requerir que los pipelines de Windows finalicen con éxito.
- Requerir revisiones aprobadas obligatorias.

8.2 Dependabot, Secret Scanning y Push Protection

Configura `.github/dependabot.yml` para auditar paquetes NuGet y npm semanalmente:



```
version: 2
updates:
  - package-ecosystem: "nuget"
    directory: "/"
    schedule:
      interval: "weekly"
```

The screenshot shows a dark-themed code editor window with a title bar containing three colored circles (red, yellow, green) and the text "YAML". The editor contains the following YAML code:

8.3 Análisis de Seguridad SAST con CodeQL

CodeQL analiza automáticamente vulnerabilidades de código fuente antes de fusionar PRs a producción.

8.4 Gobernanza con CODEOWNERS y Permisos

Archivo `.github/CODEOWNERS` :

```
# Mantenimiento general
* @mi-organizacion/tech-leads

# Responsables de infraestructura y scripts Windows
*.ps1 @mi-organizacion/windows-admins
/.github/workflows/ @mi-organizacion/devops-team
```

Parte IX: Scripting PowerShell con la API y Diagnóstico

9.1 Consultas a la API REST con `gh api` y `ConvertFrom-Json`

```
$repos = gh api user/repos | ConvertFrom-Json
$repos | Select-Object name, stargazers_count, private | Format-Table -AutoSize
```

¿Qué hace este comando?

Convierte la respuesta JSON de la API de GitHub en objetos de PowerShell nativos y los formatea en una tabla limpia.

9.2 Automatización con Scripts de PowerShell (.ps1)

Script para cerrar issues inactivos automáticamente:

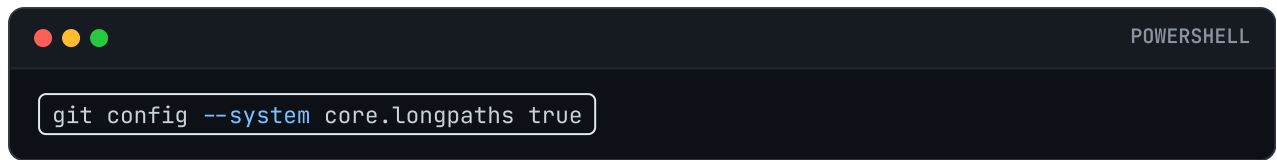
```
# cerrar-inactivos.ps1
$issues = gh issue list --label "stale" --json number,title | ConvertFrom-Json

foreach ($issue in $issues) {
    Write-Host "Cerrando issue # $($issue.number): $($issue.title)" -ForegroundColor Yellow
    gh issue close $issue.number --comment "Cerrado automáticamente por inactividad."
}
```

9.3 Diagnóstico y Resolución de Problemas Típicos de Windows

1. Filename too long en rutas de Windows

- Solución:



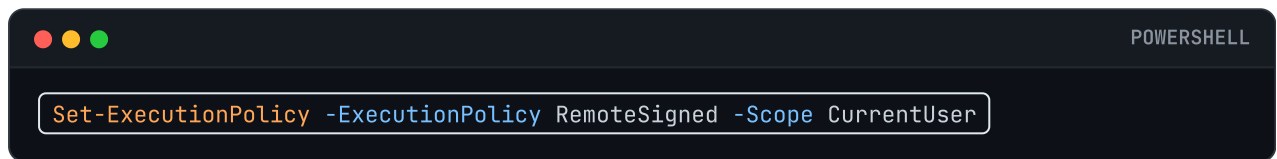
```
git config --system core.longpaths true
```

2. Archivos bloqueados por procesos en Windows

- **Causa:** Un proceso de Visual Studio, IIS o antivirus mantiene el archivo abierto.
- **Solución:** Cierra el proceso o utiliza el Administrador de Tareas antes de ejecutar `git checkout` o `git merge`.

3. Error de políticas de ejecución de scripts de PowerShell

- **Solución:**



```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```